For the `Promoter` migration, an empty string for the `famous_for` field is sufficient:

```
Please enter the default value as valid Python.
The datetime and django.utils.timezone modules are available, so it is
possible to provide e.g. timezone.now as a value.
Type 'exit' to exit this prompt
>>> ""
Migrations for 'promoters':
  promoters/migrations/0002_promoter_famous_for.py
    - Add field famous_for to promoter
```

A default value used by the migration script to populate famous_for in existing rows

The new migration file gets called 0002_promoter_famous_for.py. The name is rather helpful. It indicates that this is the second migration file to run (there are exceptions to this—more on that later), it touches the `Promoter` class, and the change is the `famous_for` field. Like with 0001_initial.py, the new migration script contains a `Migration` class declaration. This time, the `.dependencies` attribute got populated, indicating that the 0001_initial.py migration must be run before this one. This time around, the operation is `AddField`, which adds the new `famous_for` field. The complete contents of the second migration file are in the following listing.

Listing 10.5 The migration file for adding the `famous_for` field

```
# RiffMates/promoters/migrations/0002_promoter_famous_for.py
from django.db import migrations, models

class Migration(migrations.Migration):
    dependencies = [
        ("promoters", "0001_initial"),
    ]

    operations = [
        migrations.AddField(
            model_name="promoter",
            name="famous_for",
            field=models.CharField(default="", max_length=50),
            preserve_default=False,
        ),
    ]
```

This migration depends on 0001_intitial.py in the promoters app.

AddField describes the addition of the famous_for field to the database.

Now that you've seen a second migration file, hopefully, you're getting the idea of how these pieces fit together. Don't forget to run the `migration` command to invoke this latest change. If you wish, you can also populate your `famous_for` fields using the Django Admin. The migrations you've seen so far have been purely structural, but once you're using your database, any changes could also impact existing data. The next section outlines the extra work you need to do in this case.

10.2 Migration with existing data

When you added `famous_for` to the `Promoter` model, you gave Django a value for existing rows in the database. Adding content requires a little thought. *Deleting*, or moving content between columns, is a whole other challenge. Removing a field isn't